

ReactFlow Query Builder

Query Overview

This document lists every preset query shipped with the visual query builder, organised by dataset. For each query you get a one-line pipeline, a plain-English description of what it answers, and the Kafka output topic where results are published.

Contents

- Cars - Naive (19 queries)
- Cars - Optimized (24 queries)
- Volleyball - Naive (with Frame Batcher) (20 queries)

Cars - Naive

Baselines: each frame hits the LLM with no preprocessing.

Query Title	Pipeline	What it answers	Output topic
Car Brand (Generic)	Source -> Decode -> LLM -> Sink	Brand of the car in each frame, or SKIP if none.	cars_brand_generic
Car Brand Ford	Source -> Decode -> LLM -> Sink	Returns 'ford' only if the frame shows a Ford.	cars_brand_ford
Car Brand Renault	Source -> Decode -> LLM -> Sink	Returns 'renault' only if the frame shows a Renault.	cars_brand_renault
Car Brand Toyota	Source -> Decode -> LLM -> Sink	Returns 'toyota' only if the frame shows a Toyota.	cars_brand_toyota
Car Color (Generic)	Source -> Decode -> LLM -> Sink	Color of the car in each frame.	cars_color_generic
Car Color Red	Source -> Decode -> LLM -> Sink	Flags frames containing a red car.	cars_color_red
Car Color Grey	Source -> Decode -> LLM -> Sink	Flags frames containing a grey/silver car.	cars_color_grey
Car Color White	Source -> Decode -> LLM -> Sink	Flags frames containing a white car.	cars_color_white
Car Color Blue	Source -> Decode -> LLM -> Sink	Flags frames containing a blue car.	cars_color_blue
License Plates (Generic)	Source -> Decode -> LLM -> Sink	OCR of any license plate in each frame.	cars_plate_generic
Specific Plate: QRF213	Source -> Decode -> LLM -> Sink	Returns 'QRF213' only if that specific plate appears.	cars_specific_plate_qrf213
Color + License Plate (Red)	Source -> Decode -> LLM -> Sink	Plate of red cars only; SKIP otherwise.	cars_color_plate_red
Most Popular Brand	Source -> Decode -> LLM -> Window -> Aggr -> Sink	Most frequent brand in each window of frames.	cars_most_popular_brand
Most Popular Color	Source -> Decode -> LLM -> Window -> Aggr -> Sink	Most frequent color in each window.	cars_most_popular_color
Most Popular Brand and Color	Source -> Decode -> LLM -> Window -> Aggr -> Sink	Most frequent (color, brand) combination in each window.	cars_most_popular_brand_and_color
Most Popular Color (Ford)	Source -> Decode -> LLM -> Window -> Aggr -> Sink	Most frequent color among Fords seen in the window.	cars_most_popular_color_ford

Query Overview - Cars - Naive

Query Title	Pipeline	What it answers	Output topic
Most Popular Brand (Red Cars)	Source -> Decode -> LLM -> Window -> Aggr -> Sink	Most frequent brand among red cars in the window.	cars_most_popular_brand_red
Unique License Plates (Window)	Source -> Decode -> LLM -> Window -> Aggr -> Sink	Distinct plates observed per window.	cars_unique_plates_window
Repeating License Plates	Source -> Decode -> LLM -> Window -> FilterCount(>=3) -> Aggr(distinct) -> Sink	Plates that recur at least 3 times in a window.	cars_repeating_plates

Cars - Optimized

Resize and frame-skipping optimisations applied to the naive baselines.

Query Title	Pipeline	What it answers	Output topic
Car Brand (Resize)	Source -> Decode -> Resize -> LLM -> Sink	Per-frame brand; resizing cuts upstream cost.	cars_brand_resize
Car Brand (Skip 10)	Source -> Decode -> SkipFrames(10) -> LLM -> Sink	Per-frame brand; only every 10th frame reaches the LLM.	cars_brand_skip10
Car Color (Resize)	Source -> Decode -> Resize -> LLM -> Sink	Per-frame color; resized first to save bandwidth.	cars_color_resize
Car Color (Skip 10)	Source -> Decode -> SkipFrames(10) -> LLM -> Sink	Per-frame color; sampled 1 in 10 frames.	cars_color_skip10
License Plates (Resize)	Source -> Decode -> Resize -> LLM -> Sink	Plate OCR with resized input.	cars_plate_resize
License Plates (Skip 10)	Source -> Decode -> SkipFrames(10) -> LLM -> Sink	Plate OCR on 1-in-10 frames.	cars_plate_skip10
License Plates (Grayscale)	Source -> Decode -> Grayscale -> Resize -> LLM -> Sink	Plate OCR after grayscale+resize preprocessing.	cars_plate_grayscale
Specific Plate (Resize)	Source -> Decode -> Resize -> LLM -> Sink	Matches QRF213 after resize.	cars_specific_plate_resize
Specific Plate (Skip 10)	Source -> Decode -> SkipFrames(10) -> LLM -> Sink	Matches QRF213 on 1-in-10 frames.	cars_specific_plate_skip10
Specific Plate (Grayscale)	Source -> Decode -> Grayscale -> Resize -> LLM -> Sink	Matches QRF213 after grayscale+resize.	cars_specific_plate_grayscale
Color + Plate (Resize)	Source -> Decode -> Resize -> LLM -> Sink	Plate of red cars; resized first.	cars_color_plate_resize
Color + Plate (Skip 10)	Source -> Decode -> SkipFrames(10) -> LLM -> Sink	Plate of red cars; 1-in-10 frames.	cars_color_plate_skip10
Most Popular Brand (Resize)	Source -> Decode -> Resize -> LLM -> Window -> Aggr -> Sink	Top brand per window with resize preprocessing.	cars_most_popular_brand_resize
Most Popular Brand (Skip 10)	Source -> Decode -> SkipFrames(10) -> LLM -> Window -> Aggr -> Sink	Top brand per window with frame skipping.	cars_most_popular_brand_skip10
Most Popular Color (Resize)	Source -> Decode -> Resize -> LLM -> Window -> Aggr -> Sink	Top color per window with resize preprocessing.	cars_most_popular_color_resize

Query Overview - Cars - Optimized

Query Title	Pipeline	What it answers	Output topic
Most Popular Color (Skip 10)	Source -> Decode -> SkipFrames(10) -> LLM -> Window -> Aggr -> Sink	Top color per window with frame skipping.	cars_most_popular_color_skip10
Most Popular Brand+Color (Resize)	Source -> Decode -> Resize -> LLM -> Window -> Aggr -> Sink	Top (color, brand) per window with resize.	cars_most_popular_brand_color_resize
Most Popular Brand+Color (Skip 10)	Source -> Decode -> SkipFrames(10) -> LLM -> Window -> Aggr -> Sink	Top (color, brand) per window with frame skipping.	cars_most_popular_brand_color_skip10
Unique Plates (Resize)	Source -> Decode -> Resize -> LLM -> Window -> Aggr -> Sink	Distinct plates per window after resize.	cars_unique_plates_resize
Unique Plates (Skip 10)	Source -> Decode -> SkipFrames(10) -> LLM -> Window -> Aggr -> Sink	Distinct plates per window with frame skipping.	cars_unique_plates_skip10
Unique Plates (Grayscale)	Source -> Decode -> Grayscale -> Resize -> LLM -> Window -> Aggr -> Sink	Distinct plates per window after grayscale+resize.	cars_unique_plates_grayscale
Repeating Plates (Resize)	Source -> Decode -> Resize -> LLM -> Window -> FilterCount(>=3) -> Aggr(distinct) -> Sink	Plates repeating 3+ times per window, after resize.	cars_repeating_plates_resize
Repeating Plates (Skip 10)	Source -> Decode -> SkipFrames(10) -> LLM -> Window -> FilterCount(>=3) -> Aggr(distinct) -> Sink	Plates repeating 3+ times per window, with frame skipping.	cars_repeating_plates_skip10
Repeating Plates (Grayscale)	Source -> Decode -> Grayscale -> Resize -> LLM -> Window -> FilterCount(>=3) -> Aggr(distinct) -> Sink	Plates repeating 3+ times per window, after grayscale+resize.	cars_repeating_plates_grayscale

Volleyball - Naive (with Frame Batcher)

Temporal volleyball queries. Every pipeline collects 4 consecutive frames via a Frame Batcher node and sends them together in a single multi-image LLM call, giving the model temporal context for action recognition.

Query Title	Pipeline	What it answers	Output topic
Repeated Spikers (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> FilterCount(>=3) -> Aggr(distinct) -> Sink	Players who appear spiking in 3+ batches within a window.	volleyball_repeated_spikers
Repeated Setters (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> FilterCount(>=3) -> Aggr(distinct) -> Sink	Players who appear setting in 3+ batches within a window.	volleyball_repeated_setters
Repeated Blockers (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> FilterCount(>=3) -> Aggr(distinct) -> Sink	Players who appear blocking in 3+ batches within a window.	volleyball_repeated_blockers
Repeated Diggers (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> FilterCount(>=3) -> Aggr(distinct) -> Sink	Players who appear digging in 3+ batches within a window.	volleyball_repeated_diggers
Motion Category (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Per window: fraction of players moving (ALL/MOST/SOME/NONE).	volleyball_motion_category
Players Moving (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Flags batches where at least one player is moving.	volleyball_players_moving
Players Standing (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Flags batches where at least one player is standing still.	volleyball_players_standing
Players Jumping (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Flags batches where at least one player is jumping.	volleyball_players_jumping
Top Actions (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Most frequent player action per window (e.g. spiking/setting).	volleyball_top_actions
Spike Count (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Number of batches showing a spike per window.	volleyball_spike_count
Set Count (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Number of batches showing a set per window.	volleyball_set_count

Query Overview - Volleyball - Naive (with Frame Batcher)

Query Title	Pipeline	What it answers	Output topic
Block Count (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Number of batches showing a block per window.	volleyball_block_count
Set -> Spike Events (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Detects set-then-spike sequences (set, spike, other labels).	volleyball_set_spike
Set -> Block Events (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Detects set-then-block sequences.	volleyball_set_block
Dig -> Set Events (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Detects dig-then-set sequences.	volleyball_dig_set
Jump -> Spike Events (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Detects jump-then-spike sequences.	volleyball_jump_spike
Spike Bounding Boxes (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Bounding box (x1,y1,x2,y2) of the spiking player if any.	volleyball_bbox_spike
Block Bounding Boxes (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Bounding box of the blocking player if any.	volleyball_bbox_block
Set Bounding Boxes (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Bounding box of the setting player if any.	volleyball_bbox_set
Dig Bounding Boxes (Window)	Source -> Decode -> FrameBatcher(4) -> LLM -> Window(10) -> Aggr -> Sink	Bounding box of the digging player if any.	volleyball_bbox_dig